

INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Pós-Graduação em Sistemas Computacionais Embebidos
Sistemas Operativos de Tempo-Real (PGSCE-SIOPTR)

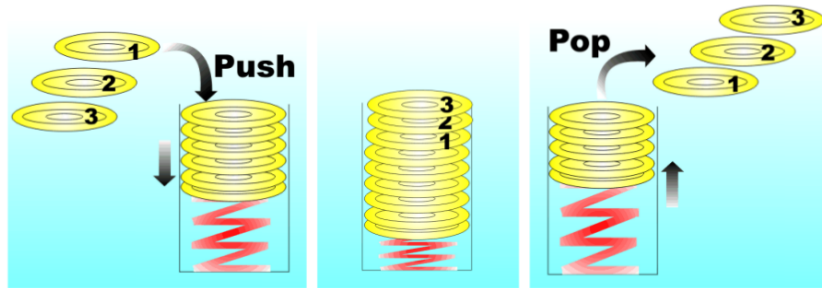
SIOPTR: TT8: Week 5

My Own KERnel (MOKER) version: Framework Scheduling

Paulo Baltarejo Sousa
pbs@isep.ipp.pt
2025/26

1 LIFO Scheduling Algorithm

It will be added a new scheduling policy to the MOKER framework. The scheduling algorithm is the well known Last-In First-Out (LIFO). According to LIFO scheduling algorithm, the last task ready for execution execution will be selected to be executed. That is, whenever a task arrives (become ready for execution) the currently executing is preempted and the recently arrived task is selected for execution.



2 Configuration Option Entry

It must be created the `MOKER_SCHED_LIFO_POLICY` configuration entry that depends on `MOKER_FRAMEWORK` entry.

Open the `linux-6.19.9-moker/kernel/moker/Kconfig` file and update it by adding:

```
menu "MOKER framework"
config MOKER_FRAMEWORK
bool "My Own KERnel Framework"
default y

config MOKER_TRACING
bool "MOKER tracing"
default y
depends on MOKER_FRAMEWORK

config MOKER_SCHED_LIFO_POLICY
bool "MOKER scheduling policy: LIFO"
default y
depends on MOKER_FRAMEWORK
endmenu
```

3 Create the Policy Identifier

We have to define a macro to identify the scheduling policy. For that purpose, it must be checked the already defined scheduling policy identifiers. They are defined in the `linux-6.19.9-moker/include/uapi/linux/sched.h` file.

```
/* SPDX-License-Identifier: GPL-2.0 WITH Linux-syscall-note */
#ifndef _UAPI_LINUX_SCHED_H
#define _UAPI_LINUX_SCHED_H
...
/*
 * Scheduling policies
 */
#define SCHED_NORMAL 0
#define SCHED_FIFO 1
#define SCHED_RR 2
#define SCHED_BATCH 3
/* SCHED_ISO: reserved but not implemented yet */
#define SCHED_IDLE 5
#define SCHED_DEADLINE 6
#define SCHED_EXT 7
...

#endif /* _UAPI_LINUX_SCHED_H */
```

Create a new macro called `SCHED_LIFO` with value 8 and add it to `linux-6.19.9-moker/include/uapi/linux/sched.h` file.

```
/* SPDX-License-Identifier: GPL-2.0 WITH Linux-syscall-note */
#ifndef _UAPI_LINUX_SCHED_H
#define _UAPI_LINUX_SCHED_H
...
/*
 * Scheduling policies
 */
#define SCHED_NORMAL 0
#define SCHED_FIFO 1
#define SCHED_RR 2
#define SCHED_BATCH 3
/* SCHED_ISO: reserved but not implemented yet */
#define SCHED_IDLE 5
#define SCHED_DEADLINE 6
#define SCHED_EXT 7

#ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
#define SCHED_LIFO 8
#endif
...

#endif /* _UAPI_LINUX_SCHED_H */
```

4 SCHED_LIFO Policy Run-queue

Each processor holds a run-queue of all runnable processes assigned to it. The scheduling policy uses this run-queue to select the "best" process to be

executed. Therefore, the run-queue is the starting point for many scheduling actions. Note, however, that processes are not directly managed by the general elements of the run-queue. This is the responsibility of the individual scheduling classes, and a class-specific sub run-queue is therefore embedded in each processor run-queue. The information for these processes is stored in a per-processor data structure called `struct rq`, which is defined in the `linux-6.19.9-moker/kernel/sched/sched.h`.

```
/*
 * This is the main, per-CPU runqueue data structure.
 */
/* Locking rule: those places that want to lock multiple runqueues
 * (such as the load balancing or the thread migration code), lock
 * acquire operations must be ordered by ascending @runqueue.
 */
struct rq {
    /* runqueue lock: */
    raw_spinlock_t __lock;

    ...

    struct cfs_rq cfs;
    struct rt_rq rt;
    struct dl_rq dl;

    ...

#ifdef CONFIG_SCHED_PROXY_EXEC
    struct task_struct __rcu *donor; /* Scheduler context */
    struct task_struct __rcu *curr; /* Execution context */
#else
    union {
        struct task_struct __rcu *donor; /* Scheduler context */
        struct task_struct __rcu *curr; /* Execution context */
    };
#endif
    ...

    /* CPU of this runqueue: */
    int cpu;
    ...
};
```

As it can be seen from the snippet code there is on sub-run-queue for CFS, RT and DL, `struct cfs_rq cfs`, `struct rt_rq rt` and `struct dl_rq dl`, respectively.

Using the same approach, it is defined a new data structure, called `struct lf_rq` for the new scheduling policy. For that purpose, create a new file called `lf_rq.h` in the `linux-6.19.9-moker/kernel/moker` directory. The content of `lf_rq.h` is:

```

#ifndef __LF_RQ_H_
#define __LF_RQ_H_

#include <linux/sched.h>
#include <linux/list.h>
#include <linux/spinlock.h>

struct lf_rq{
    struct list_head tasks;
    struct task_struct *task;
    raw_spinlock_t lock;
    unsigned nr_running;
};

void init_lf_rq(struct lf_rq *rq);
#endif

```

A stack is the appropriated container for implementing the LIFO policy. In Linux it can be easily implemented using the List API (defined in `linux-6.19.9-moker/include/linux/list.h`). The head of the stack and also the nodes have to be of `struct list_head tasks` data structure. Being the entry point, in this `struct lf_rq` data structure it is defined the head of the list. In order to protect list operations, it is also defined a `spinlock_t` variable. For optimization purpose, it is also defined a `struct task_struct` pointer that will be used to, at any instant, point to the highest priority task (that is, the most arriving recent task). It is also defined the `init_lf_rq` prototype function, which purpose is to initialize the `struct lf_rq` data structure fields.

Put this data structure visible to all kernel, by updating the `linux-6.19.9-moker/kernel/sched/sched.h`

```

...
#include "../workqueue_internal.h"

struct rq;
struct cfs_rq;
struct rt_rq;
struct sched_group;
struct cpuidle_state;

#ifdef CONFIG_PARAVIRT
# include <asm/paravirt.h>
# include <asm/paravirt_api_clock.h>
#endif

#include <asm/barrier.h>

#include "cpupri.h"
#include "cpudeadline.h"

#ifdef CONFIG_MOKER_TRACING
#include "../moker/trace.h"
#endif

#ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
# include "../moker/lf_rq.h"
#endif

/* task_struct::on_rq states: */
#define TASK_ON_RQ_QUEUED 1
#define TASK_ON_RQ_MIGRATING 2
---
```

So, the next step is to add `struct lf_rq` field to the `struct rq` data structure (defined in `linux-6.19.9-moker/kernel/sched/sched.h`).

```

struct rq {
    /* runqueue lock: */
    raw_spinlock_t __lock;

    ...

    struct cfs_rq cfs;
    struct rt_rq rt;
    struct dl_rq dl;
#ifdef CONFIG_SCHED_CLASS_EXT
    struct scx_rq scx;
#endif

#ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
    struct lf_rq lf;
#endif

    struct sched_dl_entity fair_server;

#ifdef CONFIG_FAIR_GROUP_SCHED
    /* list of leaf cfs_rq on this CPU: */
    struct list_head leaf_cfs_rq_list;
    struct list_head *tmp_alone_branch;
#endif /* CONFIG_FAIR_GROUP_SCHED */
    ...
};
```

The `raw_spinlock_t` lock as well as `struct list_head` tasks variables need to be initialized. This must be done before the Linux kernel scheduler

starts operating. For that purpose, create `lf_rq.c` file (in `linux-6.19.9-moker/kernel/moker` directory and implement `init_lf_rq` function. As it can be seen from the snippet code, all `struct lf_rq` data structure fields are initialized.

```
#include "lf_rq.h"

void init_lf_rq(struct lf_rq *rq){
    INIT_LIST_HEAD(&rq->tasks);
    raw_spin_lock_init(&rq->lock);
    rq->task = NULL;
    rq->nr_running = 0;
}
```

The best place for invoking `init_lf_rq` function, is in the `sched_init` function implemented in the `linux-6.19.9-moker/kernel/sched/core.c` file. `sched_init` function creates the CPU run queues, scheduler related data structures and initializes them.

```
void __init sched_init(void)
{
    ...
    for_each_possible_cpu(i) {
        struct rq *rq;

        rq = cpu_rq(i);
        raw_spin_lock_init(&rq->_lock);
        rq->nr_running = 0;
        rq->calc_load_active = 0;
        rq->calc_load_update = jiffies + LOAD_FREQ;
        init_cfs_rq(&rq->cfs);

#ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
        init_lf_rq(&rq->lf);
#endif

        init_rt_rq(&rq->rt);
        init_dl_rq(&rq->dl);

#ifdef CONFIG_FAIR_GROUP_SCHED
        INIT_LIST_HEAD(&rq->leaf_cfs_rq_list);
        rq->tmp_alone_branch = &rq->leaf_cfs_rq_list;
        ...
        zalloc_cpumask_var_node(&rq->scratch_mask, GFP_KERNEL, cpu_to_node(i));
    }
    set_load_weight(&init_task, false);
    init_task.se.slice = sysctl_sched_base_slice;
    ...
}
```

5 SCHED_LIFO Policy Task

A Linux process is an instance of a program in execution. To manage processes, the kernel maintains information about each process in a pro-

cess descriptor. The information stored in each process descriptor (`struct task_struct`, defined in `linux-6.19.9-moker/include/linux/sched.h`) concerns with the run-state of a process, its address space, the list of open files, the process priority and its scheduling class, just to mention some.

Since, it was created a new list in the run-queue for storing ready LIFO tasks. Recall, it was added a new run-queue for LIFO tasks, called `struct lf_rq` data structure. Such data structure stores its tasks in a double linked list, which head is the `struct list_head tasks` field. So, in order to store LIFO tasks in such a field, it is required to add a new (`struct list_head` field to the `struct task_struct`. For that purpose, create a new data structure called `struct sched_lf_entity`. The `struct sched_lf_entity` has only one field that is required to be used in the list, which head, called `tasks`, which it is defined in the `linux-6.19.9-moker/kernel/moker/lf_rq.h` file.

```
struct lf_rq{
    struct list_head tasks;
    struct task_struct *task;
    raw_spinlock_t lock;
    unsigned nr_running;
};
```

In the `linux-6.19.9-moker/kernel/moker` directory create a new file called `lf_task.h` and define the `struct sched_lf_entity` data structure in such a file:

```
#ifndef __LF_TASK_H_
#define __LF_TASK_H_

#include <linux/list.h>

struct sched_lf_entity{
    struct list_head node;
};
#endif
```

Put this data structure visible to all kernel, by updating the `linux-6.19.9-moker/include/linux/sched.h`


```

/* SPDX-License-Identifier: GPL-2.0 */
#ifndef _LINUX_SCHED_H
#define _LINUX_SCHED_H
/*
 * Define 'struct task_struct' and provide the main scheduler
 * APIs (schedule(), wakeup variants, etc.)
 */

#include <uapi/linux/sched.h>
...
#include <linux/uidgid_types.h>
#include <linux/tracepoint-defs.h>
#include <linux/unwind_deferred_types.h>
#include <asm/kmap_size.h>
#ifndef COMPILE_OFFSETS
#include <generated/rq-offsets.h>
#endif

#ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
#include "../kernel/moker/lf_task.h"
#endif

/* task_struct member predeclarations (sorted alphabetically): */
...
#endif

```

Next step is to add a new `struct sched_lf_entity` field, called `lf` to the `struct task_struct` data structure (defined in `linux-6.19.9-moker/include/linux/sched.h`).

```

struct task_struct {
#ifdef CONFIG_THREAD_INFO_IN_TASK
/*
 * For reasons of header soup (see current_thread_info()), this
 * must be the first element of task_struct.
 */
struct thread_info thread_info;
#endif
unsigned int __state;
...
int on_cpu;
...
int on_rq;
int prio;
int static_prio;
int normal_prio;
unsigned int rt_priority;

struct sched_entity se;

#ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
struct sched_lf_entity lf;
#endif

struct sched_rt_entity rt;
struct sched_dl_entity dl;
struct sched_dl_entity *dl_server;
#ifdef CONFIG_SCHED_CLASS_EXT
struct sched_ext_entity scx;
#endif
const struct sched_class *sched_class;

#ifdef CONFIG_SCHED_CORE
struct rb_node core_node;
unsigned long core_cookie;
unsigned int core_occupation;
#endif
...
#endif

```

6 SCHED_LIFO Scheduling Class

Scheduling classes are implemented using the `sched_class` data structure, which contains hooks to functions that must be called whenever a scheduling event occurs and it is defined in `linux-6.19.9-moker/kernel/sched/sched.h` file.

```

struct sched_class {
    /*
     * idle: 0
     * ext: 1
     * fair: 2
     * rt: 4
     * dl: 8
     * stop: 16
     */
    unsigned int queue_mask;

    /*
     * move_queued_task/activate_task/enqueue_task: rq->lock
     * ttwu_do_activate/activate_task/enqueue_task: rq->lock
     * wake_up_new_task/activate_task/enqueue_task: task_rq_lock
     * ttwu_runnable/enqueue_task: task_rq_lock
     * proxy_task_current: rq->lock
     * sched_change_end
     */
    void (*enqueue_task) (struct rq *rq, struct task_struct *p, int flags);
    /*
     * move_queued_task/deactivate_task/dequeue_task: rq->lock
     * __schedule/block_task/dequeue_task: rq->lock
     * proxy_task_current: rq->lock
     * wait_task_inactive: task_rq_lock
     * sched_change_begin
     */
    bool (*dequeue_task) (struct rq *rq, struct task_struct *p, int flags);
    ...
    /*
     * move_queued_task: rq->lock
     * __migrate_swap_task: rq->lock
     * ttwu_do_activate: rq->lock
     * ttwu_runnable: task_rq_lock
     * wake_up_new_task: task_rq_lock
     */
    void (*wakeup_preempt) (struct rq *rq, struct task_struct *p, int flags);
    ...
    /*
     * schedule/pick_next_task: rq->lock
     */
    struct task_struct *(*pick_task) (struct rq *rq, struct rq_flags *rf);
    ...
    /*
     * hrtick: rq->lock
     * sched_tick: rq->lock
     * sched_tick_remote: rq->lock
     */
    void (*task_tick) (struct rq *rq, struct task_struct *p, int queued);
    ...
    /*
     * sched_change
     */
    void (*switching_from) (struct rq *this_rq, struct task_struct *task);
    void (*switched_from) (struct rq *this_rq, struct task_struct *task);
    void (*switching_to) (struct rq *this_rq, struct task_struct *task);
    void (*switched_to) (struct rq *this_rq, struct task_struct *task);
    u64 (*get_prio) (struct rq *this_rq, struct task_struct *task);
    void (*prio_changed) (struct rq *this_rq, struct task_struct *task,
        u64 oldprio);
    ...
};

```

Further, each scheduling class contains a bit mask (`unsigned int queue_mask;`) that indicates its position in the hierarchy of scheduling classes.

Highest Lowest	STOP	queue_mask = 16 // 0b00010000
	Deadline (DL)	queue_mask = 8 // 0b00001000
	Real Time (RT)	queue_mask = 4 // 0b00000100
	Completely Fair Scheduler (CFS)	queue_mask = 2 // 0b00000010
	Extensible Scheduler Class (EXT)	queue_mask = 1 // 0b00000001
	Idle	queue_mask = 0 // 0b00000000

The SCHED_LIFO policy will be implemented into a new scheduling class, called Last In First Out (LIFO) scheduling class. Such class will be positioned between RT and DL scheduling classes.

Highest Lowest	STOP	queue_mask = 32 // 0b00100000
	Deadline (DL)	queue_mask = 16 // 0b00010000
	Last In First Out (LIFO)	queue_mask = 8 // 0b00001000
	Real Time (RT)	queue_mask = 4 // 0b00000100
	Completely Fair Scheduler (CFS)	queue_mask = 2 // 0b00000010
	Extensible Scheduler Class (EXT)	queue_mask = 1 // 0b00000001
	Idle	queue_mask = 0 // 0b00000000

Any scheduling class must be implemented as an instance of `sched_class` data structure.

For creating the LIFO scheduling class, create a new file called `sched.c` in the `linux-6.19.9-moker/kernel/moker` directory and define the `lf_sched_class` variable using `DEFINE_SCHED_CLASS` macro (`linux-6.19.9-moker/kernel/sched/sched.h`).

```
#include "../sched/sched.h"

/*
 * LIFO scheduling class.
 * Implements SCHED_LIFO
 */

static void enqueue_task_lf(struct rq *rq, struct task_struct *p, int flags)
{
}
```

```

static bool dequeue_task_lf(struct rq *rq, struct task_struct *p, int flags)
{
    return true;
}

/*
 * Preempt the current task with a newly woken task if needed:
 */
static void wakeup_preempt_lf(struct rq *rq, struct task_struct *p, int flags)
{
}

static struct task_struct *pick_task_lf(struct rq *rq, struct rq_flags *rf)
{
    return NULL;
}

static void put_prev_task_lf(struct rq *rq, struct task_struct *p, struct task_struct *next)
{
}

static void set_next_task_lf(struct rq *rq, struct task_struct *p, bool first)
{
}

static int select_task_rq_lf(struct task_struct *p, int cpu, int flags)
{
    return cpu;
}

static void task_tick_lf(struct rq *rq, struct task_struct *p, int queued)
{
}

static void prio_changed_lf(struct rq *rq, struct task_struct *p, u64 oldprio)
{
}

static void switched_to_lf(struct rq *rq, struct task_struct *p)
{
}

static void update_curr_lf(struct rq *rq)
{
}

DEFINE_SCHED_CLASS(lf) = {

    .queue_mask    = 8,

    .enqueue_task  = enqueue_task_lf,
    .dequeue_task  = dequeue_task_lf,

    .wakeup_preempt = wakeup_preempt_lf,

    .pick_task     = pick_task_lf,
    .put_prev_task = put_prev_task_lf,
    .set_next_task = set_next_task_lf,

    .select_task_rq = select_task_rq_lf,
    .set_cpus_allowed = set_cpus_allowed_common,

    .task_tick     = task_tick_lf,

    .prio_changed  = prio_changed_lf,
    .switched_to   = switched_to_lf,
    .update_curr   = update_curr_lf,
};

```

Recall, the `lf_sched_class` scheduling policy it is positioned between the RT and CFS scheduling classes. Then, for that the `lf_sched_class` class

variable has to be positioned between `dl_sched_class` and `rt_sched_class` scheduling class variables. This is done by the `SCHED_DATA` macro defined into `linux-6.19.9-moker/include/asm-generic/vmlinux.lds.h` file.

```

...
/*
 * The order of the sched class addresses are important, as they are
 * used to determine the order of the priority of each sched class in
 * relation to each other.
 */

#ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
#define SCHED_DATA \
    STRUCT_ALIGN(); \
    __sched_class_highest = .; \
    *(__stop_sched_class) \
    *(__dl_sched_class) \
    *(__lf_sched_class) \
    *(__rt_sched_class) \
    *(__fair_sched_class) \
    *(__idle_sched_class) \
    __sched_class_lowest = .;
#else
#define SCHED_DATA \
    STRUCT_ALIGN(); \
    __sched_class_highest = .; \
    *(__stop_sched_class) \
    *(__dl_sched_class) \
    *(__rt_sched_class) \
    *(__fair_sched_class) \
    *(__idle_sched_class) \
    __sched_class_lowest = .;
#endif

/* The actual configuration determine if the init/exit sections
 * are handled as text/data or they can be discarded (which
 * often happens at runtime)
 */
...
```

Ensure that the `lf_sched_class` scheduling class is correctly placed into the scheduling classes array. Make the following change to the `sched_init` function in the `linux-6.19.9-moker/kernel/sched/core.c` file.

```

...
void __init sched_init(void)
{
    unsigned long ptr = 0;
    int i;

    /* Make sure the linker didn't screw up */
    BUG_ON(!sched_class_above(&stop_sched_class, &dl_sched_class));
    BUG_ON(!sched_class_above(&dl_sched_class, &rt_sched_class));
#ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
    BUG_ON(!sched_class_above(&dl_sched_class, &lf_sched_class));
    BUG_ON(!sched_class_above(&lf_sched_class, &rt_sched_class));
#endif
    BUG_ON(!sched_class_above(&rt_sched_class, &fair_sched_class));
    BUG_ON(!sched_class_above(&fair_sched_class, &idle_sched_class));
#ifdef CONFIG_SCHED_CLASS_EXT
    BUG_ON(!sched_class_above(&fair_sched_class, &ext_sched_class));
    BUG_ON(!sched_class_above(&ext_sched_class, &idle_sched_class));
#endif

    wait_bit_init();
    ...
}

```

Given the position of the `lf_sched_class` scheduling class, the `queue_mask` value of the higher priority scheduling classes must be changed.

Make the following change to the `sched_init` function in the `linux-6.19.9-moker/kernel/sched/stop_task.c` file.

```

/*
 * Simple, special scheduling class for the per-CPU stop tasks:
 */
DEFINE_SCHED_CLASS(stop) = {

#ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
    .queue_mask = 32,
#else
    .queue_mask = 16,
#endif

    .enqueue_task = enqueue_task_stop,
    ...
};

```

Make the following change to the `sched_init` function in the `linux-6.19.9-moker/kernel/sched/deadline.c` file.

```

DEFINE_SCHED_CLASS(dl) = {
    @2
#ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
    .queue_mask = 16,
#else
    .queue_mask = 8,
#endif

    .enqueue_task = enqueue_task_dl,
    ...
};

```

Let the kernel knows the `lf_sched_class` `sched_class` variable. For that

purpose, update the linux-6.19.9-moker/kernel/sched/sched.h file.

```
...
#define DEFINE_SCHED_CLASS(name) \
const struct sched_class name##_sched_class \
__aligned(__alignof__(struct sched_class)) \
__section("__" #name "_sched_class")

/* Defined in include/asm-generic/vmlinux.lds.h */
extern struct sched_class __sched_class_highest[];
extern struct sched_class __sched_class_lowest[];

extern const struct sched_class stop_sched_class;
extern const struct sched_class dl_sched_class;

#ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
extern const struct sched_class lf_sched_class;
#endif

extern const struct sched_class rt_sched_class;
extern const struct sched_class fair_sched_class;
extern const struct sched_class idle_sched_class;
...
```

Implement the SCHED_LIFO scheduling algorithm requires coding four lf_sched_class functions: enqueue_task_lf, dequeue_task_lf, wakeup_preempt_lf and pick_task_lf. However, the others must be implemented but with no code. Update linux-6.19.9-moker/kernel/moker/sched.c file with:

```
#include "../sched/sched.h"

/*
 * LIFO scheduling class.
 * Implements SCHED_LIFO
 */

static void enqueue_task_lf(struct rq *rq, struct task_struct *p, int flags)
{
    raw_spin_lock(&rq->lf.lock);
    list_add(&p->lf.node, &rq->lf.tasks);
    rq->lf.task = p;
    rq->lf.nr_running++;
    add_nr_running(rq, 1);
    raw_spin_unlock(&rq->lf.lock);
}

static bool dequeue_task_lf(struct rq *rq, struct task_struct *p, int flags)
{
    struct sched_lf_entity *t = NULL;
    raw_spin_lock(&rq->lf.lock);
    list_del(&p->lf.node);
    if(list_empty(&rq->lf.tasks)){
        rq->lf.task = NULL;
    }else{
        t = list_first_entry(&rq->lf.tasks, struct sched_lf_entity, node);
        rq->lf.task = container_of(t, struct task_struct, lf);
    }
}
```



```

    rq->lf.nr_running--;
    sub_nr_running(rq, 1);
    raw_spin_unlock(&rq->lf.lock);

    return true;
}

static void wakeup_preempt_lf(struct rq *rq, struct task_struct *p, int flags)
{
    switch(rq->donor->policy){
        case SCHED_DEADLINE:
            break;
        case SCHED_FIFO:
        case SCHED_RR:
        case SCHED_NORMAL:
        case SCHED_BATCH:
        case SCHED_IDLE:
            //case SCHED_RESET_ON_FORK:
        case SCHED_LIFO:
            resched_curr(rq);
            break;
    }
}

static struct task_struct *pick_task_lf(struct rq *rq, struct rq_flags *rf)
{
    struct task_struct * p = NULL;
    raw_spin_lock(&rq->lf.lock);
    p = rq->lf.task;
    raw_spin_unlock(&rq->lf.lock);
    return p;
}
...

```

7 Associating LIFO tasks to SCHED_LIFO scheduling class

Right now, it was defined a LIFO task and a scheduling class to schedule LIFO tasks. Next step, it is associate LIFO tasks to the `lf_sched_class` scheduling class.

Highest ↑ ↓ Lowest	STOP	
	Deadline (DL)	SCHED_DEADLINE
	Last In First Out (LIFO)	SCHED_LIFO
	Real Time (RT)	SCHED_FIFO, SCHED_RR
	Completely Fair Scheduler (CFS)	SCHED_NORMAL, SCHED_BATCH, SCHED_IDLE
	Extensible Scheduler Class (EXT)	SCHED_EXT
	Idle	

This is done by invoking the `sched_setscheduler` system call (number 144, in `linux-6.19.9-moker/arch/x86/entry/syscalls/syscall_64.tbl`).

```
# SPDX-License-Identifier: GPL-2.0 WITH Linux-syscall-note
#
# 64-bit system call numbers and entry vectors
#
# The format is:
# <number> <abi> <name> <entry point> [<compat entry point> [noreturn]]
#
# The __x64_sys_*() stubs are created on-the-fly for sys_*() system calls
#
# The abi is "common", "64" or "x32" for this file.
#
...
142 common sched_setparam sys_sched_setparam
143 common sched_getparam sys_sched_getparam
144 common sched_setscheduler sys_sched_setscheduler
145 common sched_getscheduler sys_sched_getscheduler
146 common sched_get_priority_max sys_sched_get_priority_max
147 common sched_get_priority_min sys_sched_get_priority_min
148 common sched_rr_get_interval sys_sched_rr_get_interval
149 common mlock sys_mlock
```

This function is coded in the `linux-6.19.9-moker/kernel/sched/syscalls.c` file and eventually will call `sched_setscheduler` function.

```

...
static int do_sched_setscheduler(pid_t pid, int policy, struct sched_param __user *param)
{
    struct sched_param lparam;

    if (unlikely(!param || pid < 0))
        return -EINVAL;
    if (copy_from_user(&lparam, param, sizeof(struct sched_param)))
        return -EFAULT;

    CLASS(find_get_task, p)(pid);
    if (!p)
        return -ESRCH;

    return sched_setscheduler(p, policy, &lparam);
}
...
SYSCALL_DEFINE3(sched_setscheduler, pid_t, pid, int, policy, struct sched_param __user *, param)
{
    if (policy < 0)
        return -EINVAL;

    return do_sched_setscheduler(pid, policy, param);
}

```

From the `sched_setscheduler` function it is invoked the `_sched_setscheduler` function, which in turn calls `__sched_setscheduler` function.

In this function are invoked two functions that needs to be updated to support LIFO tasks, which are: `valid_policy` and `__setscheduler_class`.

```

int __sched_setscheduler(struct task_struct *p,
const struct sched_attr *attr,
bool user, bool pi)
{
    int oldpolicy = -1, policy = attr->sched_policy;
    int retval, oldprio, newprio;
    const struct sched_class *prev_class, *next_class;
    struct balance_callback *head;
    struct rq_flags rf;
    int reset_on_fork;
    int queue_flags = DEQUEUE_SAVE | DEQUEUE_MOVE | DEQUEUE_NOCLOCK;
    struct rq *rq;
    bool cpuset_locked = false;

    /* The pi code expects interrupts enabled */
    BUG_ON(pi && in_interrupt());
recheck:
    /* Double check policy once rq lock held: */
    if (policy < 0) {
        reset_on_fork = p->sched_reset_on_fork;
        policy = oldpolicy = p->policy;
    } else {
        reset_on_fork = !(attr->sched_flags & SCHED_FLAG_RESET_ON_FORK);

        if (!valid_policy(policy))
            return -EINVAL;
    }
    ...
    prev_class = p->sched_class;
    next_class = __setscheduler_class(policy, newprio);
    ...}

```

The `valid_policy` function it is implemented in `linux-6.19.9-moker/kernel/sched/sched.h` file. Update this file in order to accept `SCHED_LIFO` policy.

```
...
static inline int idle_policy(int policy)
{
    return policy == SCHED_IDLE;
}

static inline int normal_policy(int policy)
{
#ifdef CONFIG_SCHED_CLASS_EXT
    if (policy == SCHED_EXT)
        return true;
#endif
    return policy == SCHED_NORMAL;
}

static inline int fair_policy(int policy)
{
    return normal_policy(policy) || policy == SCHED_BATCH;
}

static inline int rt_policy(int policy)
{
    return policy == SCHED_FIFO || policy == SCHED_RR;
}

#ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
static inline int lf_policy(int policy)
{
    return policy == SCHED_LIFO;
}
#endif

static inline int dl_policy(int policy)
{
    return policy == SCHED_DEADLINE;
}

static inline bool valid_policy(int policy)
{
    return idle_policy(policy) || fair_policy(policy) ||
        rt_policy(policy) || dl_policy(policy)
#ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
        || lf_policy(policy)
#endif
}
;
...
```

The `__setscheduler_class` function it is implemented `linux-6.19.9-moker/kernel/sched/core.c` file. Update this function in order to assign LIFO tasks to the `lf_sched_class` scheduling class.

```

...
const struct sched_class *__setscheduler_class(int policy, int prio)
{
    if (dl_prio(prio))
        return &dl_sched_class;

#ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
    if (lf_policy(policy))
        return &lf_sched_class;
#endif

    if (rt_prio(prio))
        return &rt_sched_class;

#ifdef CONFIG_SCHED_CLASS_EXT
    if (task_should_scx(policy))
        return &ext_sched_class;
#endif

    return &fair_sched_class;
}
...

```

Since the LIFO scheduling algorithm depends on the enqueue order, so it is relevant that such information being collected. For that purpose update `linux-6.19.9-moker/kernel/moker/trace.h` and `linux-6.19.9-moker/kernel/moker/trace.c` files to support such events.

```

#ifdef __TRACE_H_
#define __TRACE_H_
...
enum evt{
    MISC_EVT = 0,
    SCHED_TICK,
    SWITCH_AWAY,
    SWITCH_TO,
    ENQUEUE_RQ,
    DEQUEUE_RQ
};
...
#endif

```

```

...
static int dequeue (char *buffer){
    int ret = 0, len;
    char evt[10];
    spin_lock(&trace.lock);
    if(!is_empty(trace.read_item,trace.write_item)){ //if it is not empty

        switch((int)trace.events[trace.read_item].event){
            case MISC_EVT:
                strcpy(evt,"MC_EVT");
                break;
            case SCHED_TICK:
                strcpy(evt,"SCH_TK");
                break;
            case SWITCH_AWAY:
                strcpy(evt,"SWT_AY");
                break;
            case SWITCH_TO:
                strcpy(evt,"SWT_TO");
                break;
            case ENQUEUE_RQ:
                strcpy(evt,"ENQ_RQ");
                break;
            case DEQUEUE_RQ:
                strcpy(evt,"DEQ_RQ");
                break;
            default:
                strcpy(evt,"UK_EVT");
        }

        len = sprintf(buffer,"%llu,",trace.events[trace.read_item].time);
        len += sprintf(buffer + len,"%s,",evt);
        len += sprintf(buffer+len,"%d*",(int)trace.events[trace.read_item].number);
        len += sprintf(buffer+len,"%d,", (int)trace.events[trace.read_item].policy);
        len += sprintf(buffer+len,"%d,", (int)trace.events[trace.read_item].prio);
        len += sprintf(buffer+len,"%d,", (int)trace.events[trace.read_item].pid);
        len += sprintf(buffer+len,"%d,", (int)trace.events[trace.read_item].state);
        len += sprintf(buffer+len,"%s\n",trace.events[trace.read_item].comm);

        increment(&trace.read_item);
        ret = 1;
    }
    spin_unlock(&trace.lock);
    return ret;
}
...

```

Update linux-6.19.9-moker/kernel/moker/sched.c file to collect data from such events.

```

#include "../sched/sched.h"

/*
 * LIFO scheduling class.
 * Implements SCHED_LIFO
 */

static void enqueue_task_lf(struct rq *rq, struct task_struct *p, int flags)
{
    ...
    raw_spin_unlock(&rq->lf.lock);

#ifdef CONFIG_MOKER_TRACING
    moker_trace(ENQUEUE_RQ, -1, p);
#endif
}

static void dequeue_task_lf(struct rq *rq, struct task_struct *p, int flags)
{
    ...
    raw_spin_unlock(&rq->lf.lock);

#ifdef CONFIG_MOKER_TRACING
    moker_trace(DEQUEUE_RQ, -1, p);
#endif

    return true;
}
...

```

For optimization purpose, change the MOKER tracing mechanism for collecting only SCHED_LIFO events. So, update linux-6.19.9-moker/kernel/sched/core.c file.

```

static void __sched notrace __schedule(bool preempt)
{
    ...

    trace_sched_switch(preempt, prev, next);

#ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
    if(lf_policy(prev->policy) || lf_policy(next->policy)){
#endif

#ifdef CONFIG_MOKER_TRACING
        moker_trace(SWITCH_AWAY, -1, prev);
        moker_trace(SWITCH_TO, -1, next);
#endif

#ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
    }
#endif

    /* Also unlocks the rq: */
    rq = context_switch(rq, prev, next, &rf);
} else {
    rq->clock_update_flags &= ~(RQCF_ACT_SKIP|RQCF_REQ_SKIP);
    rq_unlock_irq(rq, &rf);
}

    balance_callback(rq);
}

```

Since, the scheduler tick event is not important for LIFO scheduling algorithm let us ignore it by updating the `sched_tick` function (`linux-6.19.9-moker/kernel/sched/core.c`)

```
void sched_tick(void)
{
    ...

    /*
     * ifdef CONFIG_MOKER_TRACING
     * moker_trace(SCHED_TICK, -1, rq->donor);
     * endif
     */
    ...
}
```

8 Eliminating the CFS Starvation Mechanism

The Linux kernel implements a mechanism designed to solve the long-standing issue of Real-Time (RT) Throttling. It is a "fake" DL task that ensures standard processes (CFS tasks) are not completely starved of CPU time by runaway RT tasks. Being a DL task, it runs within the DL class—which has a higher priority than LIFO tasks and the RT tasks. In order to prevent this mechanism, update `linux-6.19.9-moker/kernel/sched/fair.c` file with:

```
...
/*
 * The enqueue_task method is called before nr_running is
 * increased. Here we update the fair scheduling stats and
 * then put the task into the rbtree:
 */
static void
enqueue_task_fair(struct rq *rq, struct task_struct *p, int flags)
{
    struct cfs_rq *cfs_rq;
    struct sched_entity *se = &p->se;
    int h_nr_idle = task_has_idle_policy(p);
    int h_nr_runnable = 1;
    int task_new = !(flags & ENQUEUE_WAKEUP);
    int rq_h_nr_queued = rq->cfs.h_nr_queued;
    u64 slice = 0;

    ...

    ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
    //this prevents the CFS starvation mechanism
    rq_h_nr_queued = 1;
    endif
    if (!rq_h_nr_queued && rq->cfs.h_nr_queued)
        dl_server_start(&rq->fair_server);
    ...
}
...
```


9 Compiling

For compiling all this code update the `linux-6.19.9-moker/kernel/moker/Makefile` file

```
# MOKER framework makefile

obj-$(CONFIG_MOKER_TRACING) += trace.o
obj-y+=syscalls.o

obj-$(CONFIG_MOKER_SCHED_LIFO_POLICY) += sched.o lf_rq.o
```

Compile the Linux kernel:

```
sudo ./kcompile.sh
```

```
sudo reboot
```

10 Testing

Download `tasks.tar.gz` file from https://drive.google.com/file/d/1gg7C1Fh28XYy_zBsUBL7QvAYu9TEGJx/view?usp=sharing. This compressed file contains three source code files (`def.h`, `task.c` and `launcher.c`) a `Makefile` file and task set file configuration (`taskset.txt`). By executing the `make` it creates two programs: `launcher` and `task`.

The `launcher` reads the task set configuration file, `taskset.txt`, and for each line it forks a process for executing `task`. Here it is an configuration file example. This file is composed by three lines, which means that three `task` processes will be created. The content of each line is: `task_id`, `C`, `T`, `O`, `njobs`. Columns `C`, `T` and `O` are set in nanoseconds. The purpose of each parameter will be explained next.

```
1,100000000000,600000000000,200000000,2,
2,200000000000,600000000000,100000000,2,
3,800000000000,1200000000000,000000000,1,
```

The `task` is recurrent, it is based on loop. For each loop iteration (activation) it executes (do some work), next it sleeps until a specific timestamp, then it wakes up.

Each `task` receives from `launcher` six parameters: `task_id`, `C`, `T`, `O`, `time0` and `njobs`.

The `task_id` is the logical `task` identifier. The `C` is the amount of time that task takes to make its work. The `T` is period of each activation (loop

iteration). The `O` is the offset of the first activation. The `time0` is the time instant from which the time starts. Finally, the `njobs` is the number of activations, that is, the number of loop iterations.

```
...
int main(int argc, char** argv)
{
    struct sched_param param;
    unsigned long long C, T, O, time0, release;
    unsigned int task_id, njobs, i=0;

    struct timespec r;

    task_id=atoi(argv[1]);
    C=(unsigned long long)atoll(argv[2]);
    T=(unsigned long long)atoll(argv[3]);
    O=(unsigned long long)atoll(argv[4])+OFFSET;
    time0 = (unsigned long long)atoll(argv[5]);
    njobs=atoi(argv[6]);

    printf("Task(%d,%d): before SCHED_LIFO\n",task_id,getpid());
    param.sched_priority = 0;
    if((sched_setscheduler(0,SCHED_LIFO,&param)) == -1){
        perror("ERROR:sched_setscheduler failed");
        exit(-1);
    }
    printf("Task(%d,%d): after SCHED_LIFO\n",task_id,getpid());

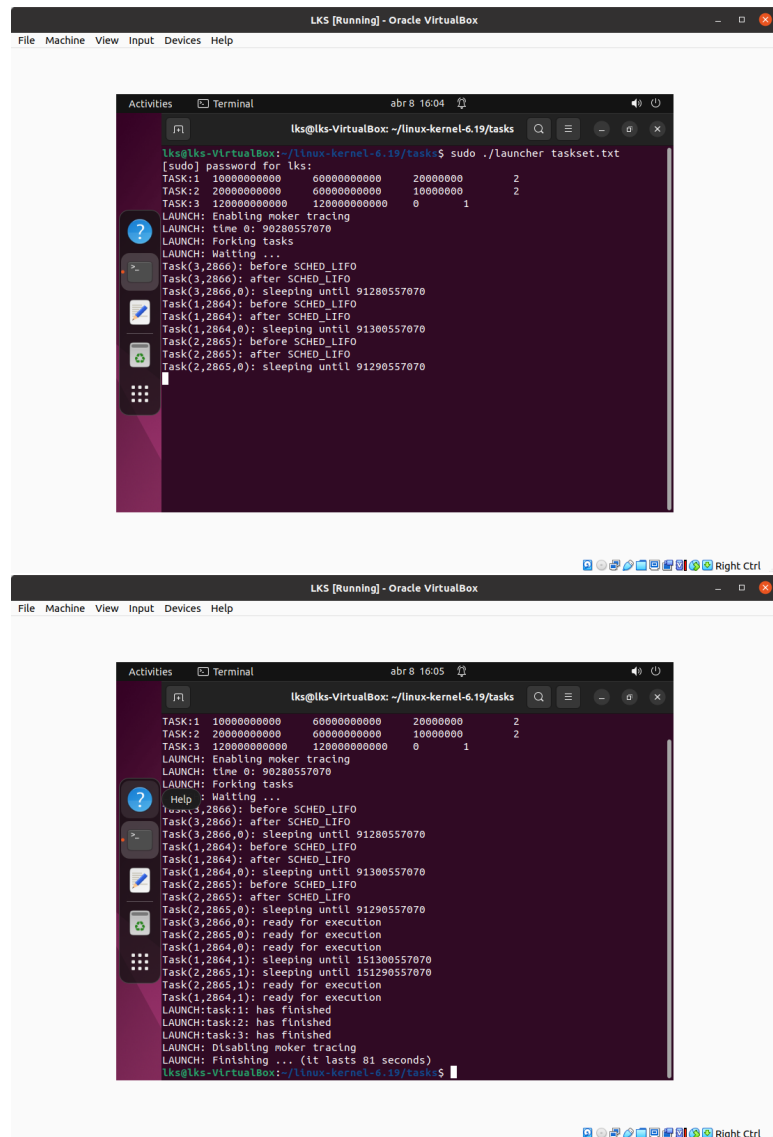
    release = time0 + O;
    for(i=0;i<njobs;i++){
        r.tv_sec = release / NSEC_PER_SEC;
        r.tv_nsec = release % NSEC_PER_SEC;

        printf("Task(%d,%d,%d): sleeping until %lld\n",task_id,getpid(),i,release);
        clock_nanosleep(CLOCK_MONOTONIC,TIMER_ABSTIME, &r,NULL);
        printf("Task(%d,%d,%d): ready for execution\n",task_id,getpid(),i);

        do_work(C);

        //computes the next release
        release += T;
    }
    exit(task_id);
    return 0;
}
```

1. Extract it by typing:
`tar -xzvf tasks.tar.gz`
2. Compile it by invoking:
`make`
3. Put the set of tasks running:
`sudo ./launcher taskset.txt`



4. Wait (it takes approximately 90 seconds) and get the scheduling trace:
`cat /proc/moker_trace > trace.csv`

5. Check if the scheduling output is according to the LIFO algorithm.
`gedit trace.csv`

Recall, this tracing mechanism outputs a set of data separated by comma. Each line is formatted as:

<timestamp>, <event>, <*number*>, <policy>, <prio>, <pid>, <state>, <comm>

The first column refers to the timestamp and second one to the event type. The following columns present task's (or process) data:

- Scheduling policy (**policy**)

```
#define SCHED_NORMAL 0
#define SCHED_FIFO 1
#define SCHED_RR 2
...
#define SCHED_LIFO 8
```

- Priority level (**prio**)
- The process identifier(**pid**)
- Process state (**state**)

```
#define TASK_RUNNING 0x00000000 //0
#define TASK_INTERRUPTIBLE 0x00000001 // 1
#define TASK_DEAD 0x00000080 // 128
define TASK_WAKING 0x00000200 // 512
#define TASK_FREEZABLE 0x00002000 // 8192

TASK_FREEZABLE | TASK_INTERRUPTIBLE 0x00002001 // 8193
```

- Process name (**comm**)
 - The process executable name

```

85281333473,ENQ_RQ,*-1*,8,120,2866,0,task
85281337497,DEQ_RQ,*-1*,8,120,2866,8193,task
85281338509,SWT_AY,*-1*,8,120,2866,8193,task
85281338544,SWT_TO,*-1*,0,120,2864,0,launcher
85281521093,ENQ_RQ,*-1*,8,120,2864,0,task
85281523832,DEQ_RQ,*-1*,8,120,2864,8193,task
85281524326,SWT_AY,*-1*,8,120,2864,8193,task
85281524360,SWT_TO,*-1*,0,120,2865,0,launcher
85281697406,ENQ_RQ,*-1*,8,120,2865,0,task
85281700151,DEQ_RQ,*-1*,8,120,2865,8193,task
85281700760,SWT_AY,*-1*,8,120,2865,8193,task
85281700794,SWT_TO,*-1*,0,120,12,0,kworker/u4:0
91280680115,ENQ_RQ,*-1*,8,120,2866,512,task
91280706008,SWT_AY,*-1*,0,120,0,0,swapper/0
91280706418,SWT_TO,*-1*,8,120,2866,0,task
91296357610,ENQ_RQ,*-1*,8,120,2865,512,task
91296382886,SWT_AY,*-1*,8,120,2866,0,task
91296382914,SWT_TO,*-1*,8,120,2865,0,task
91314431646,ENQ_RQ,*-1*,8,120,2864,512,task
91314433736,SWT_AY,*-1*,8,120,2865,0,task
91314433777,SWT_TO,*-1*,8,120,2864,0,task
95248006301,DEQ_RQ,*-1*,8,120,2864,8193,task
95248008616,SWT_AY,*-1*,8,120,2864,8193,task
95248008652,SWT_TO,*-1*,8,120,2865,0,task
103309227694,DEQ_RQ,*-1*,8,120,2865,8193,task
103309229725,SWT_AY,*-1*,8,120,2865,8193,task
103309229758,SWT_TO,*-1*,8,120,2866,0,task
151294546297,ENQ_RQ,*-1*,8,120,2865,512,task
151294554111,SWT_AY,*-1*,8,120,2866,0,task
151294554149,SWT_TO,*-1*,8,120,2865,0,task
151300591737,ENQ_RQ,*-1*,8,120,2864,512,task
151300594384,SWT_AY,*-1*,8,120,2865,0,task
151300594429,SWT_TO,*-1*,8,120,2864,0,task
156620829018,DEQ_RQ,*-1*,8,120,2864,128,task
156620830811,SWT_AY,*-1*,8,120,2864,128,task
156620830841,SWT_TO,*-1*,8,120,2865,0,task
167041227942,DEQ_RQ,*-1*,8,120,2865,128,task
167041229796,SWT_AY,*-1*,8,120,2865,128,task
167041229837,SWT_TO,*-1*,8,120,2866,0,task
171490336232,DEQ_RQ,*-1*,8,120,2866,128,task
171490339546,SWT_AY,*-1*,8,120,2866,128,task
171490339604,SWT_TO,*-1*,2,0,1895,0,rtkit-daemon

```